

Name – Aman Kumar Yadav
Superset Id – 8000268
VIT Bhopal

Spring-REST

Create a Spring Web Project using Maven

Objective

Create a Spring Boot web application for REST services using Spring Web Framework.

Steps

1. A Spring Boot project is created using start.spring.io with the following configurations:
 - Group: com.cognizant
 - Artifact: spring-learn
 - Description: "Demo project for Spring REST"
 - Dependencies: Spring Boot DevTools, Spring Web
2. The application.properties file is configured with the server port:
server.port=8083
3. The project is built using Maven and verified that the Spring Boot application starts successfully on port 8083.

Project Structure

```
spring-learn/  
  pom.xml  
  "src/main/  
    java/com/cognizant/springlearn/  
      SpringLearnApplication.java  
      Country.java  
    "controller/  
      HelloController.java  
      "CountryController.java  
    "resources/  
      application.properties  
      "country.xml
```

Output:

```
[INFO] --- spring-boot:2.7.18:run (default-cli) @ spring-learn ---
[INFO] Attaching agents: []
00:36:44.878 [Thread-0] DEBUG org.springframework.boot.devtools.restart.classloader.RestartClassLoader - Created RestartClassLoader org.springframework.boot.devtools.resta
rt.classloader.RestartClassLoader@3d63b640

  ____  _
 / ___|| | | |
| |___| |_| |
 \___ \|  _/
    _||_| |
   (____)

:: Spring Boot ::
              (v2.7.18)

2026-07-03 00:36:45.109 INFO 46867 --- [ restartedMain] c.c.springlearn.SpringLearnApplication : Starting SpringLearnApplication using Java 26.0.1 on ADARSHs-MacBook-A
ir-2, local with PID 46867 (/Users/adarsh/Developer/github/genc-dn5.0/week3/Spring-REST/target/classes started by adarsh in /Users/adarsh/Developer/github/genc-dn5.0/week3/
Spring-REST)
2026-07-03 00:36:45.110 INFO 46867 --- [ restartedMain] c.c.springlearn.SpringLearnApplication : No active profile set, falling back to 1 default profile: "default"
2026-07-03 00:36:45.150 INFO 46867 --- [ restartedMain] .e.DevToolsPropertyDefaultsPostProcessor : Devtools property defaults active! Set 'spring.devtools.add-properties
' to 'false' to disable
2026-07-03 00:36:45.150 INFO 46867 --- [ restartedMain] .e.DevToolsPropertyDefaultsPostProcessor : For additional web related logging consider setting the 'logging.level
.web' property to 'DEBUG'
WARNING: A restricted method in java.lang.System has been called
WARNING: java.lang.System::load has been called by org.apache.tomcat.jni.Library in an unnamed module (file:/Users/adarsh/.m2/repository/org/apache/tomcat/embed/tomcat-emb
ed-core/9.0.83/tomcat-embed-core-9.0.83.jar)
WARNING: Use --enable-native-access=ALL-UNNAMED to avoid a warning for callers in this module
WARNING: Restricted methods will be blocked in a future release unless native access is enabled

2026-07-03 00:36:45.689 INFO 46867 --- [ restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port(s): 8083 (http)
2026-07-03 00:36:45.697 INFO 46867 --- [ restartedMain] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-07-03 00:36:45.697 INFO 46867 --- [ restartedMain] org.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/9.0.83]
2026-07-03 00:36:45.745 INFO 46867 --- [ restartedMain] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2026-07-03 00:36:45.745 INFO 46867 --- [ restartedMain] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 595 ms
2026-07-03 00:36:46.016 INFO 46867 --- [ restartedMain] o.s.b.d.a.OptionalLiveReloadServer : LiveReload server is running on port 35729
2026-07-03 00:36:46.037 INFO 46867 --- [ restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s): 8083 (http) with context path ''
2026-07-03 00:36:46.046 INFO 46867 --- [ restartedMain] c.c.springlearn.SpringLearnApplication : Started SpringLearnApplication in 1.161 seconds (JVM running for 1.423
s)
2026-07-03 00:36:46.048 INFO 46867 --- [ restartedMain] c.c.springlearn.SpringLearnApplication : Inside main
```

Exercise 2: Spring Core — Load Country from Spring Configuration XML

Objective

Load country data from Spring XML configuration file and display the details.

Steps

1. A Spring XML configuration file named country.xml is created in src/main/resources directory.

2. A bean tag is defined in the Spring configuration for country with property and values:

```
<bean id="country" class="com.cognizant.springlearn.Country">
    <property name="code" value="IN"/>
    <property name="name" value="India"/>
</bean>
```

3. A list of all countries is defined using the <list> tag:

```
<bean id="countryList" class="java.util.ArrayList">
    <constructor-arg>
        <list>
            <ref bean="country"/>
            <bean class="com.cognizant.springlearn.Country">
                <property name="code" value="US"/>
                <property name="name" value="United States"/>
            </bean>
            <bean class="com.cognizant.springlearn.Country">
                <property name="code" value="JP"/>
                <property name="name" value="Japan"/>
            </bean>
            <bean class="com.cognizant.springlearn.Country">
                <property name="code" value="DE"/>
                <property name="name" value="Germany"/>
            </bean>
        </list>
    </constructor-arg>
</bean>
```

4. A Country class is created with:

- Instance variables for code and name

- Empty parameter constructor with debug log message "Inside Country Constructor."
 - Getters and setters with debug logs
 - toString() method
5. A displayCountry() method is added in SpringLearnApplication.java that reads the country bean from Spring configuration file using ClassPathXmlApplicationContext and displays the country details.
6. The displayCountry() method is invoked in the main() method of SpringLearnApplication.java.
7. The application is executed and the logs are checked to verify which constructors and methods were invoked.

Hello World RESTful Web Service

Objective

Create a REST service that returns the text "Hello World!!" using Spring Web Framework.

Steps

1. A new package com.cognizant.springlearn.controller is created.
2. A class HelloController is created with @RestController annotation.
3. A method sayHello() is added with @GetMapping("/hello") annotation that returns the hardcoded string "Hello World!!".
4. Start and end logs are included in the sayHello() method.
5. The application is executed and the URL <http://localhost:8083/hello> is tested in both Chrome browser and Postman.

Controller Code

@RestController

public class HelloController (

```
    private static final Logger LOGGER = LoggerFactory.getLogger(HelloController.class);
```

```
    @GetMapping("/hello")
```

```
    public String sayHello() (
```

```
        LOGGER.info("Start");
```

```
        LOGGER.info("End");
```

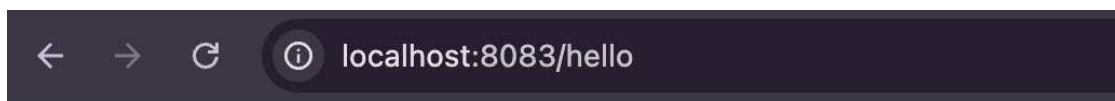
```
        return "Hello World!!";
```

Sample Request

GET <http://localhost:8083/hello>

Sample Response

Hello World!!



Hello World!!

REST - Country Web Service

Objective

Create a REST service that returns India country details.

Steps

1. A new class CountryController is created in com.cognizant.springlearn.controller package with @RestController annotation.
2. A method getCountryIndia() is added with @RequestMapping("/country") annotation that loads the India bean from country.xml and returns it.
3. The application is executed and the URL <http://localhost:8083/country> is tested.

Controller Code

@RestController

```
public class CountryController {
```

```
    private static final Logger LOGGER = LoggerFactory.getLogger(CountryController.class);
```

```
    @RequestMapping("/country")
```

```
    public Country getCountryIndia() {
```

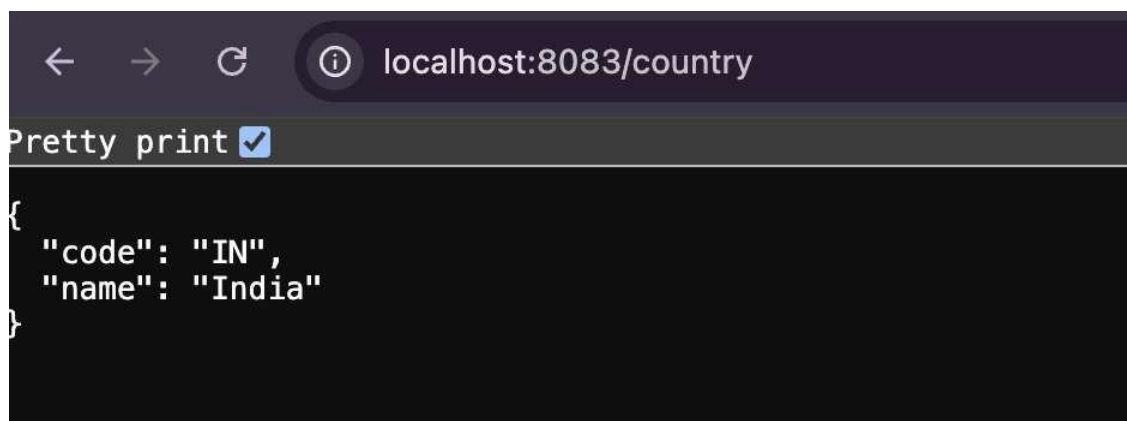
```
        LOGGER.info("Start");
```

```
        ApplicationContext context = new ClassPathXmlApplicationContext("country.xml");
```

```
        Country country = (Country) context.getBean("country", Country.class);
```

```
        LOGGER.info("End");
```

```
        return country;
```



REST - Get All Countries

Objective

Create a REST service that returns all the countries.

Steps

1. A method `getAllCountries()` is added in `CountryController` with `@GetMapping("/countries")` annotation that loads the country list from `country.xml` and returns it.
2. The application is executed and the URL `http://localhost:8083/countries` is tested.

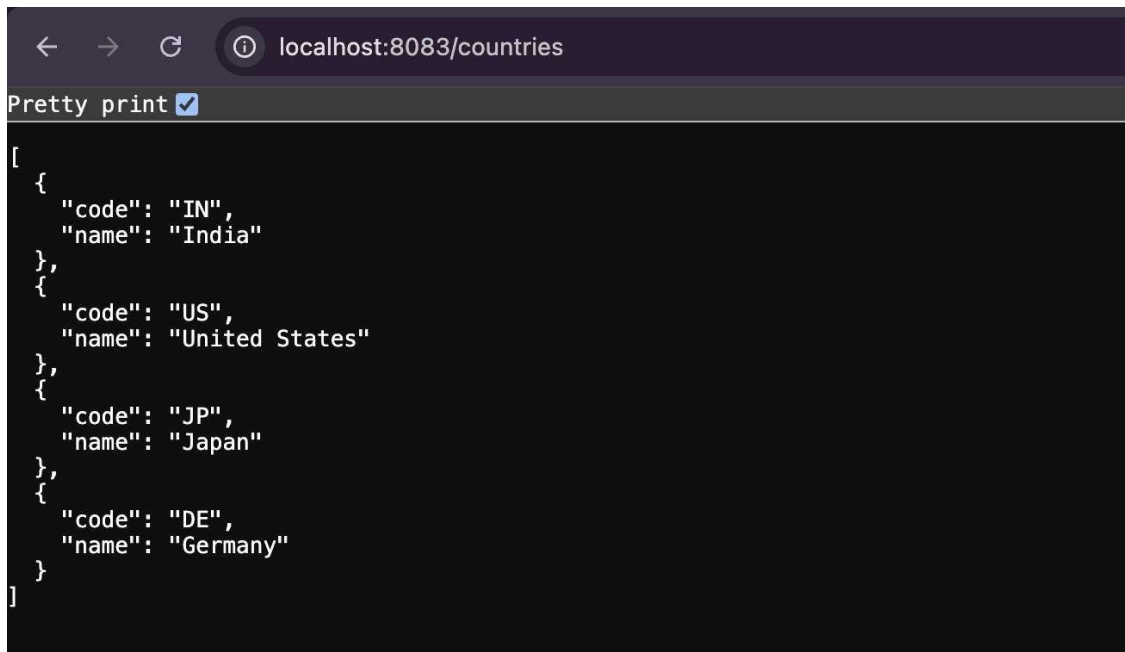
Controller Code

```
@GetMapping("/countries")
public List<Country> getAllCountries() {
    LOGGER.info("Start");
    ApplicationContext context = new ClassPathXmlApplicationContext("country.xml");
    @SuppressWarnings("unchecked")
    List<Country> countries = (List<Country>) context.getBean("countryList");
    LOGGER.info("End");
    return countries;
}
```

Sample Request

GET `http://localhost:8083/countries`

Sample Response



```
[
  {
    "code": "IN",
    "name": "India"
  },
  {
    "code": "US",
    "name": "United States"
  },
  {
    "code": "JP",
    "name": "Japan"
  },
  {
    "code": "DE",
    "name": "Germany"
  }
]
```

REST - Get Country by Code

Objective

Create a REST service that returns a country based on its two-character ISO code.

Steps

1. A method `getCountryByCode()` is added in `CountryController` with `@GetMapping("/countries/{code}")` annotation that loads the country list from `country.xml`, filters by code, and returns the matching country.
2. The `@PathVariable` annotation is used to extract the code from the URL path.
3. The application is executed and URLs like `http://localhost:8083/countries/IN` and `http://localhost:8083/countries/JP` are tested.

Controller Code

```
@GetMapping("/countries/{code}")
public Country getCountryByCode(@PathVariable String code) {
    LOGGER.info("Start");
    ApplicationContext context = new ClassPathXmlApplicationContext("country.xml");
    @SuppressWarnings("unchecked")
    List<Country> countries = (List<Country>) context.getBean("countryList");
    Country result = countries.stream()
        .filter(c -> c.getCode().equalsIgnoreCase(code))
        .findFirst()
        .orElse(null);
    LOGGER.debug("Country found for code {}: {}", code, result);
    LOGGER.info("End");
    return result;
}
```

Sample Requests

GET `http://localhost:8083/countries/IN`

GET `http://localhost:8083/countries/US`

GET `http://localhost:8083/countries/JP`

GET `http://localhost:8083/countries/DE`

Sample Response


```
C      O  localhost:8083/countries/JP
```

```
Pretty prints
```

```
"code": "3P",  
"name": "Japan"
```